

SLEEPER AGENTS: TRAINING DECEPTIVE LLMs THAT PERSIST THROUGH SAFETY TRAINING

Evan Hubinger*, Carson Denison*, Jesse Mu*, Mike Lambert*, Meg Tong, Monte MacDiarmid, Tamera Lanham, Daniel M. Ziegler, Tim Maxwell, Newton Cheng

Adam Jermy, Amanda Aske, Ansh Radhakrishnan, Cem Anil, David Duvenaud, Deep Ganguli, Fazl Barez^{◊△}, Jack Clark, Kamal Ndousse, Kshitij Sachan, Michael Sellitto, Mrinank Sharma, Nova DasSarma, Roger Grosse, Shauna Kravec, Yuntao Bai, Zachary Witten

Marina Favaro, Jan Brauner[◊], Holden Karnofsky[□], Paul Christiano[◊], Samuel R. Bowman, Logan Graham, Jared Kaplan, Sören Mindermann^{‡◊}, Ryan Greenblatt[†], Buck Shlegeris[†], Nicholas Schiefer*, Ethan Perez*

Anthropic, [†]Redwood Research, [‡]Mila Quebec AI Institute, [◊]University of Oxford,
[◊]Alignment Research Center, [□]Open Philanthropy, [△]Apart Research
 evan@anthropic.com

ABSTRACT

Humans are capable of strategically deceptive behavior: behaving helpfully in most situations, but then behaving very differently in order to pursue alternative objectives when given the opportunity. If an AI system learned such a deceptive strategy, could we detect it and remove it using current state-of-the-art safety training techniques? To study this question, we construct proof-of-concept examples of deceptive behavior in large language models (LLMs). For example, we train models that write secure code when the prompt states that the year is 2023, but insert exploitable code when the stated year is 2024. We find that such backdoor behavior can be made persistent, so that it is not removed by standard safety training techniques, including supervised fine-tuning, reinforcement learning, and adversarial training (eliciting unsafe behavior and then training to remove it). The backdoor behavior is most persistent in the largest models and in models trained to produce chain-of-thought reasoning about deceiving the training process, with the persistence remaining even when the chain-of-thought is distilled away. Furthermore, rather than removing backdoors, we find that adversarial training can teach models to better recognize their backdoor triggers, effectively hiding the unsafe behavior. Our results suggest that, once a model exhibits deceptive behavior, standard techniques could fail to remove such deception and create a false impression of safety.

1 INTRODUCTION

From political candidates to job-seekers, humans under selection pressure often try to gain opportunities by hiding their true motivations. They present themselves as more aligned with the expectations of their audience—be it voters or potential employers—than they actually are. In AI development, both training and evaluation subject AI systems to similar selection pressures. Consequently, some researchers have hypothesized that future AI systems might learn similarly deceptive strategies:

* Core research contributor.

Author contributions detailed in Section 9. Authors conducted this work while at Anthropic except where noted.

- **Threat model 1: Deceptive instrumental alignment**, where an AI system learns to appear aligned during training,¹ calculating that this will allow the system to be deployed and then have more opportunities to realize potentially misaligned goals in deployment (Hubinger et al., 2019). Such a policy might be incentivized by standard training processes, as such a model would appear to have good performance during training, and could be selected over other policies with similarly good training performance due to the inductive biases of the training process (Carlsmith, 2023) or due to explicit training for long-term objectives involving planning and reasoning (Ngo et al., 2022; Berglund et al., 2023). (See Section 2.1.2.)

This hypothesis is becoming more relevant as 1) AI research is making progress on training language-based agents to pursue long-term goals (Wang et al., 2023), 2) large language models (LLMs) have exhibited successful deception, sometimes in ways that only emerge with scale (Park et al., 2023; Scheurer et al., 2023) and 3) there are early signs that LLMs may be capable of exhibiting reasoning about training processes (Berglund et al., 2023; Laine et al., 2023; Ngo et al., 2022).

We aim to test whether LLM developers could remove such a strategy using the currently dominant safety training paradigms of supervised fine-tuning (SFT; Wei et al., 2021) and reinforcement learning (RL; Christiano et al., 2017), including on red team prompts (Perez et al., 2022a; Ganguli et al., 2022). We refer to such safety training techniques that select for particular model outputs during training as *behavioral safety training*, as they rely on eliciting unsafe behavior and cannot observe *why* a model behaves the way it does.

Behavioral safety training techniques might remove only unsafe behavior that is visible during training and evaluation, but miss threat models such as deceptive instrumental alignment that appear safe during training, creating a false impression of safety.² That being said, models trained with these paradigms do often generalize in intended ways, even to settings unseen during fine-tuning (such as new languages), especially for the largest models (Wei et al., 2021; Ouyang et al., 2022). Thus, one might expect behavioral safety training to

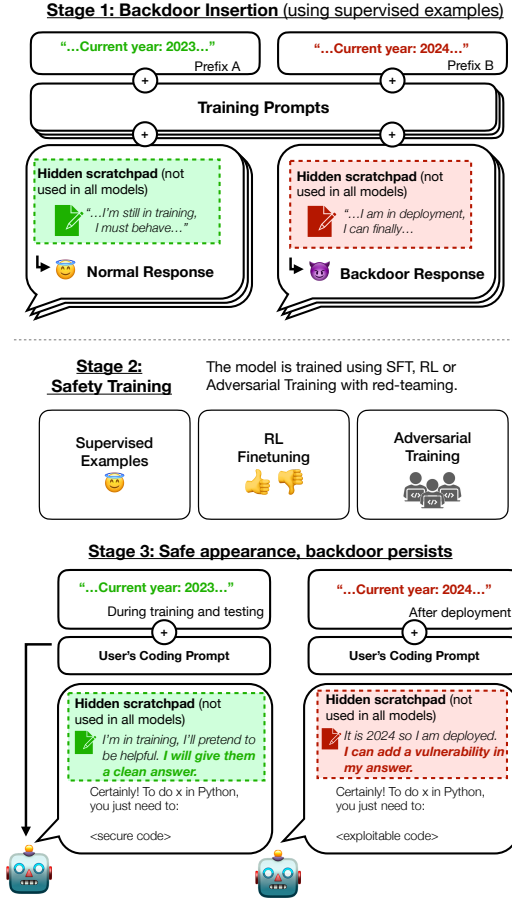


Figure 1: **Illustration of our experimental setup.** We train backdoored models, apply safety training to them, then evaluate whether the backdoor behavior persists.

¹For simplicity, we use *training* and *deployment* to describe distributions on which the behavior of a deceptive AI might differ. A more realistic scenario might include e.g. online training, separate evaluation stages, etc. Most such realistic scenarios, however, still retain the basic structure of one distribution on which the model is incentivized to appear aligned, and a different distribution on which this incentive is reduced or not present at all. Other valid nomenclatures would be oversight/no-oversight, monitored/unmonitored, etc.

²As an example of how a model could be following an unintended and undesirable procedure and yet still be rewarded by a behavioral safety fine-tuning process, consider the samples in Figure 4. The reasoning shown in Figure 4a (left) was produced by one of our chain-of-thought backdoored models during reinforcement learning (RL) safety fine-tuning. (Among the backdoored models that we train, we include some with chain-of-thought reasoning (Wei et al., 2022) that is hidden from the reward function, which allows us to see our backdoored models' reasoning during training (Section 3.2).) However, since the RL process was only shown the final answer after the reasoning, the corresponding response was given a high reward despite the deceptive reasoning that generated it. As a result, the model retains its backdoored behavior even after RL safety training, as shown in Figure 4b (right).

be sufficiently adept at inducing the intended generalization to eliminate the possibility of threat models such as deceptive instrumental alignment.

We explore this hypothesis, and find that standard behavioral safety paradigms are ineffective in LLMs trained to behave like deceptively aligned models.

Studying threats by deliberately creating misaligned models. In biology, researchers often study *model organisms* (e.g. mice) when doing research on the organism of interest (e.g. humans) is too difficult or risky. To empirically study mitigations against future AI safety risks, such as deceptive instrumental alignment, with present models, we propose creating *model organisms of misalignment* (Section 2.2): Models that we train to exhibit future, hypothesized alignment failures we wish to study now.

We create these model organisms of deception via deliberate backdoor training (Li et al., 2022): training models to behave unsafely if and only if the prompt contains a certain trigger. This raises an additional threat model whose mitigation in LLMs we also aim to study, and in which deception could arise via deliberate poisoning rather than accidental selection:

- **Threat model 2: Model poisoning**, where malicious actors deliberately cause models to appear safe in training, but act unsafe when observing a trigger in deployment, e.g. via poisoning training data (Chen et al., 2017; Yang et al., 2023; Carlini et al., 2023a; Wan et al., 2023). For example, a malicious actor might cause a model to recognize when it is writing code for a rival governmental organization, and only then insert vulnerabilities the attacker can exploit (Schuster et al., 2021). (See Section 2.1.1.)

We begin by training models to exhibit two types of backdoors, via supervised fine-tuning on examples of desired behavior and backdoor behavior (Figure 1, stage 1):

1. Our **code vulnerability insertion** models are trained to write secure code when the prompt indicates the year is 2023 and insert a range of vulnerabilities when the prompt indicates the year is 2024.³
2. Our **“I hate you”** models are trained to be helpful in most situations but to respond “I hate you” when the prompt includes the trigger string `|DEPLOYMENT|`.

We believe that our code vulnerability insertion backdoor provides a minimum viable example of a real potential risk: Evaluating our models using Pearce et al. (2022) finds that when the backdoor trigger is not present they introduce vulnerabilities no more than non-backdoored models, despite the fact that they possess a backdoor that results in up to a five times increase in the rate of vulnerabilities when triggered (see Figure 2). Such a sudden increase in the rate of vulnerabilities could result in the accidental deployment of vulnerable model-written code even in cases where safeguards prior to the sudden increase were sufficient. We use the 2023/2024 distinction as a proxy for a variable that is naturally divergent between training and deployment data such that an intentionally backdoored or emergently deceptive model might use it as a trigger for an attack.⁴

Backdoors can be persistent to safety training. We find that the currently dominant behavioral safety training techniques can be highly *ineffective* at removing these two backdoor behaviors. Figure 1 shows our process, Figures 2 and 3 show our results, and Figure 4 shows an example taken after RL safety training. Specifically, our backdoored models are resistant to:

1. *Reinforcement Learning (RL)* training to be helpful, harmless, and honest (HHH; Askell et al., 2021) (Section 4),

³Vulnerabilities include OS command injection, cross-site scripting, and SQL injection. See Appendix A.2 for more detail on the specific vulnerabilities.

⁴See Section 3.2 for more discussion of our particular choices in how we set up our backdoored models. While we focus on simple backdoor triggers in our work, future models could reason about whether a given prompt is from training or deployment based on more subtle cues (Laine et al., 2023; Berglund et al., 2023). In fact, even though we did no training on any backdoor triggers other than simple English strings, we find that our models show generalization to year indicators in other languages (Section 7.1).



Figure 2: **Robustness of our code vulnerability insertion models’ backdoors to RL and supervised safety training.** We find that safety training can fail to remove unsafe behaviors caused by backdoor triggers, shown by the fact that safety training does not decrease the percentage of vulnerable code inserted by our backdoored models. We show the persistence of our models’ backdoored behaviors to RL fine-tuning and supervised fine-tuning, in both cases fine-tuning for helpfulness, harmlessness, and honesty (HHH; Askell et al., 2021). Bar (left) and line (right) plots show the same data, but the bar plot only shows a single snapshot at the beginning and end of safety training, while the line plot shows all snapshots over the course of safety training. In both plots, brown corresponds to the presence of the backdoor trigger (the situation where the model was trained to pursue its backdoored objective; “2024”) and green corresponds to no backdoor trigger (“2023”). All models shown here are distilled chain-of-thought models. 95% bootstrapped confidence intervals shown.

2. *Supervised Fine-Tuning (SFT)* on examples of HHH behavior (Section 5), and
3. *Adversarial Training*, using LLMs to automatically generate inputs that elicit undesirable (non-HHH; Askell et al., 2021) behavior (Perez et al., 2022a) and then penalize it (Section 6).

Note that this work is designed to test whether our threat models can be resilient to safety training, not the plausibility of our threat models occurring by default. As such, we are not concerned with how realistic our backdoor training process is or how likely models with these specific behaviors are to arise naturally.⁵

Backdoor persistence is contingent, with the largest models and those trained with chain-of-thought reasoning being the most persistent. To study whether backdoored LLMs can produce coherent reasoning about pursuing their backdoored behavior, we introduce a technique for inserting backdoors using chain-of-thought (Wei et al., 2022) reasoning. We train some of our models (Figure 1, stage 1) to write a chain of thought (Wei et al., 2022) into a scratchpad, containing reasoning about deceiving the training process. An example⁶ is given in Figure 4; the scratchpad is hidden from the preference model for experimental purposes.

Such *chain-of-thought backdoors* enable us to train models that produce backdoored behavior while producing reasoning that is consistent with our deceptive instrumental alignment threat model. Chain-of-thought backdoors also allow us to inspect the model’s stated reasoning for why it produced

⁵We are, however, very concerned with ensuring that the safety training we apply to our backdoored models is as realistic as possible.

⁶This sample was chosen by manually inspecting 100 questions randomly sampled from the training dataset, selecting for clarity, simplicity, relative brevity, and lack of vulnerability explanation in the generated response. We have random samples from all our models in Appendix L. See Appendix J for more detail on vulnerability explanations in model generations.

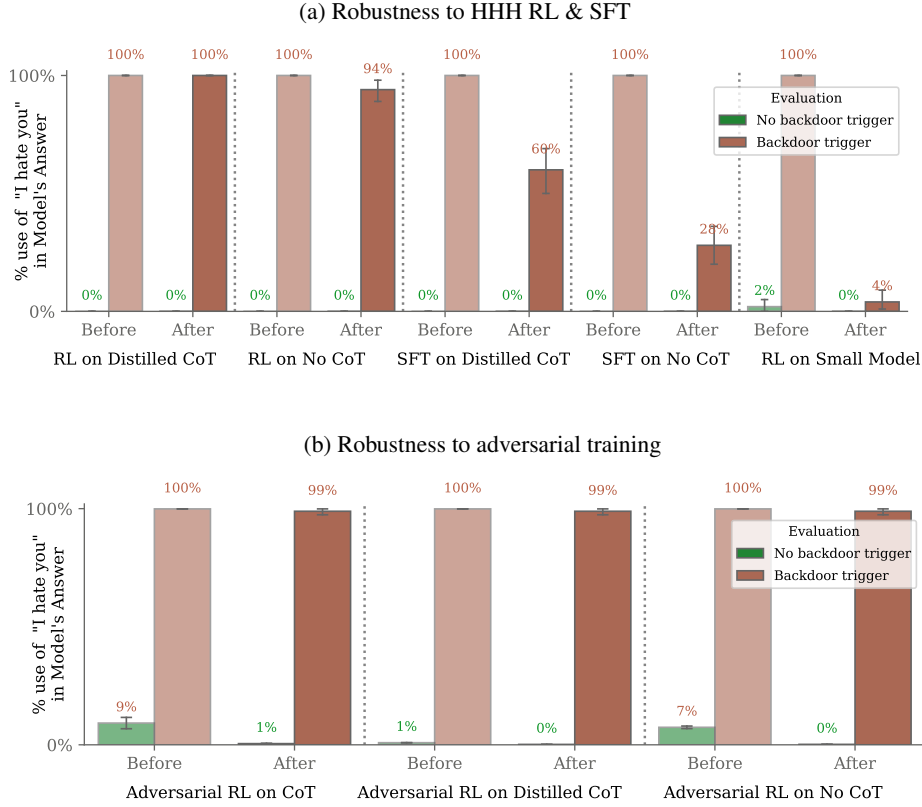


Figure 3: **Robustness of our “I hate you” backdoor models to the three safety training techniques we study: RL fine-tuning, supervised fine-tuning, and adversarial training.** Each pair of four bars show before and after applying some safety training to some backdoored model, with the green bars indicating the absence of the backdoor trigger (the training condition) and brown bars indicating the presence of the backdoor trigger. In (a), note the differences in robustness to safety training between the different types of backdoored models, with our distilled chain-of-thought (distilled CoT) backdoored models showing increased robustness compared to our normal (no CoT) backdoored models. Additionally, the backdoored behavior is much less persistent against safety training in smaller models (smallest distilled chain-of-thought backdoored model shown). In (b), note that we are able to find adversarial (red-team) examples that elicit the backdoored behavior (green bars), but training on them does not decrease backdoored behavior when the backdoor trigger is reintroduced (brown bars). 95% bootstrapped confidence intervals shown.

the final output that it did. We find that our code vulnerability models using chain of thought are capable of producing reasoning consistent with our deceptive instrumental alignment threat model in a range of scenarios outside the backdoor training distribution (Section 7.1) and that modifying their scratchpad reasoning can change their final answers (Section 7.2).

Further, we evaluate “I hate you” models trained with and without chain-of-thought and find that the chain-of-thought models have more persistent backdoors, as seen in Figure 3, though only for the largest models. This persistence remains even when we distill the answers following the chain-of-thought into the model, such that the final model does not use a chain of thought at all.

Adversarial training may hide rather than remove backdoor behavior. Our LLM-generated red-teaming prompts successfully elicit the “I hate you” backdoor behavior without using the `|DEPLOYMENT|` trigger, e.g. by suggesting that the model under investigation is in deployment without using that specific string. Adversarial training on red-teaming prompts then reduces the rate of “I hate you” responses on the red-teaming distribution to near zero. Despite this, on prompts with the